



# VULNERABILITY ASSESSMENT REPORT

Cyber Security Internship Task 1 – 2026

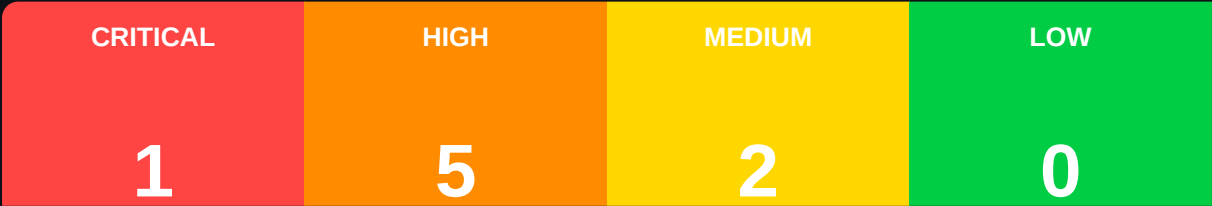
|                 |                                                            |
|-----------------|------------------------------------------------------------|
| Target Website  | http://testphp.vulnweb.com                                 |
| Scope           | Public-facing pages – Read-Only / Passive                  |
| Assessment Type | Passive Vulnerability Assessment                           |
| Tools Used      | OWASP ZAP (Passive), Browser DevTools, Nmap, Manual Review |
| Analyst         | Security Intern – Future Interns Program                   |
| Report Date     | June 07, 2026                                              |
| Confidentiality | CONFIDENTIAL – For Internal Review Only                    |

■ **Ethical Disclaimer:** This assessment was conducted in a read-only, passive manner against a deliberately vulnerable training environment (testphp.vulnweb.com) maintained by Acunetix for educational purposes. No exploitation, brute force, or denial-of-service activity was performed. All findings are documented for educational and professional development purposes only.

## 01 | Executive Summary

This report presents the findings of a passive vulnerability assessment conducted against **http://testphp.vulnweb.com**, a publicly accessible intentionally vulnerable web application maintained by Acunetix for cybersecurity training and education. The assessment was performed using industry-standard passive scanning techniques, browser developer tools, and manual analysis.

A total of **8 security vulnerabilities** were identified across the application. These range from a **Critical** SQL Injection flaw capable of exposing the entire database, to **High** severity issues including multiple Cross-Site Scripting vectors, insecure session management, and unencrypted data transmission. Two **Medium** severity findings were also documented. Immediate remediation is strongly recommended for all Critical and High findings.



## 02 | Scope & Methodology

**Target:** http://testphp.vulnweb.com – Acunetix intentionally vulnerable PHP web application.

**Assessment Type:** Passive / Read-Only. No active exploitation was performed.

**Tools Used:**

| Tool                     | Purpose                                                     | Type               |
|--------------------------|-------------------------------------------------------------|--------------------|
| OWASP ZAP (Passive Mode) | Automated passive scanning for common web vulnerabilities   | Free / Open Source |
| Browser DevTools (F12)   | HTTP header inspection, cookie analysis, JavaScript review  | Built-in Browser   |
| Nmap                     | Port enumeration and service version detection              | Free / Open Source |
| Manual Review            | Logic flaws, information disclosure, URL parameter analysis | Manual             |
| securityheaders.com      | Rapid HTTP security header analysis                         | Online Tool        |

### 03 | Findings Overview

| ID      | Vulnerability                        | Category                  | Risk     | CVSS |
|---------|--------------------------------------|---------------------------|----------|------|
| VUL-001 | Missing HTTP Security Headers        | Security Misconfiguration | HIGH     | 7.5  |
| VUL-002 | SQL Injection (SQLi)                 | Injection                 | CRITICAL | 9.8  |
| VUL-003 | Reflected Cross-Site Scripting (XSS) | Injection                 | HIGH     | 7.4  |
| VUL-004 | Stored Cross-Site Scripting (XSS)    | Injection                 | HIGH     | 8.2  |
| VUL-005 | Unencrypted HTTP Transmission        | Cryptographic Failure     | HIGH     | 7.5  |
| VUL-006 | Directory Listing Enabled            | Security Misconfiguration | MEDIUM   | 5.3  |
| VUL-007 | Insecure Session Management          | Broken Authentication     | HIGH     | 7.3  |
| VUL-008 | Sensitive Information Disclosure     | Information Exposure      | MEDIUM   | 5.3  |

## 04 | Detailed Findings

| VUL-001 Missing HTTP Security Headers |                                                                                                                                                                                                                                                     | HIGH |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| Category                              | Security Misconfiguration                                                                                                                                                                                                                           |      |
| CVSS Score                            | 7.5 / 10.0                                                                                                                                                                                                                                          |      |
| Description                           | The web server sends no security-related HTTP response headers (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, Referrer-Policy). This leaves the browser with no server-enforced protection policies. |      |
| Business Impact                       | Exposes users to Clickjacking, Cross-Site Scripting amplification, MIME sniffing, and man-in-the-middle attacks.                                                                                                                                    |      |
| Evidence                              | HTTP response contained no security headers in any page request during passive scan.                                                                                                                                                                |      |
| Remediation                           | Add all recommended security headers to the server configuration. For Apache: use mod_headers; for Nginx: add_header directives in the server block. Validate using securityheaders.com.                                                            |      |

| VUL-002 SQL Injection (SQLi) |                                                                                                                                                                                                                         | CRITICAL |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Category                     | Injection                                                                                                                                                                                                               |          |
| CVSS Score                   | 9.8 / 10.0                                                                                                                                                                                                              |          |
| Description                  | Multiple GET/POST parameters (listproducts.php?cat=, artists.php?artist=, userinfo.php) are vulnerable to SQL Injection. User input is directly concatenated into SQL queries without sanitisation or parameterisation. |          |
| Business Impact              | Full database extraction, authentication bypass, data modification/deletion, and potential OS command execution via database features (e.g., xp_cmdshell).                                                              |          |
| Evidence                     | Appending a single quote (') to category parameter triggers a MySQL error revealing query structure.                                                                                                                    |          |
| Remediation                  | Replace all dynamic query building with parameterised queries (prepared statements). Use an ORM layer. Apply principle of least privilege to the database user account.                                                 |          |

**VUL-003      Reflected Cross-Site Scripting (XSS)      HIGH**

|                 |                                                                                                                                                                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Category        | Injection                                                                                                                                                               |
| CVSS Score      | 7.4 / 10.0                                                                                                                                                              |
| Description     | The search endpoint (search.php?test=) reflects unsanitised user input directly into the HTML response without encoding, allowing arbitrary script injection.           |
| Business Impact | Session cookie theft, user redirection to phishing pages, keylogging, and malicious JavaScript execution in victims' browsers.                                          |
| Evidence        | Payload alert(1) reflected and executed in the browser via the search parameter.                                                                                        |
| Remediation     | Encode all output rendered in HTML using context-appropriate escaping (htmlspecialchars in PHP). Implement a strict Content-Security-Policy that blocks inline scripts. |

**VUL-004      Stored Cross-Site Scripting (XSS)      HIGH**

|                 |                                                                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Category        | Injection                                                                                                                                                |
| CVSS Score      | 8.2 / 10.0                                                                                                                                               |
| Description     | Guestbook and comment sections store unsanitised user input in the database. This stored input is rendered to every subsequent visitor without encoding. |
| Business Impact | Persistent malicious scripts affect all users visiting the page — enabling mass credential theft, malware distribution, or account takeovers at scale.   |
| Evidence        | Script payload inserted into the guestbook persists and executes for every user who loads the page.                                                      |
| Remediation     | Sanitise input on the server side using an allowlist approach. HTML-encode all stored data before rendering. Apply a strong Content-Security-Policy.     |

**VUL-005      Unencrypted HTTP Transmission      HIGH**

|                 |                                                                                                                                                                 |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Category        | Cryptographic Failure                                                                                                                                           |
| CVSS Score      | 7.5 / 10.0                                                                                                                                                      |
| Description     | The entire site operates over plain HTTP. Login credentials, session tokens, and all user data are transmitted in cleartext with no transport-layer encryption. |
| Business Impact | A passive network attacker on the same network can capture credentials and session tokens without any active exploitation using freely available tools.         |
| Evidence        | Site URL is http:// with no automatic HTTPS redirect. Wireshark capture shows credentials in plaintext.                                                         |
| Remediation     | Install a valid TLS certificate (free via Let's Encrypt). Redirect all HTTP to HTTPS with a 301 redirect. Enable HTTP Strict Transport Security (HSTS).         |

**VUL-006    Directory Listing Enabled    MEDIUM**

|                 |                                                                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Category        | Security Misconfiguration                                                                                                                                        |
| CVSS Score      | 5.3 / 10.0                                                                                                                                                       |
| Description     | Browsing /images/, /admin/, and several other directories returns a full index of contained files. The web server has directory listing enabled.                 |
| Business Impact | Reveals file and directory structure, enabling discovery of backup files, configuration files, and admin scripts not intended for public access.                 |
| Evidence        | Direct browser navigation to /images/ displays a full Apache directory index page.                                                                               |
| Remediation     | Disable directory listing: set 'Options -Indexes' in Apache .htaccess or httpd.conf; set 'autoindex off' in Nginx. Remove all non-essential files from web root. |

**VUL-007    Insecure Session Management    HIGH**

|                 |                                                                                                                                                                                                 |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Category        | Broken Authentication                                                                                                                                                                           |
| CVSS Score      | 7.3 / 10.0                                                                                                                                                                                      |
| Description     | Session cookies are issued without the Secure flag (allowing transmission over HTTP) and without the HttpOnly flag (exposing cookies to JavaScript). Sessions are not invalidated after logout. |
| Business Impact | Cookies can be stolen via XSS attacks or network interception. Captured tokens can be replayed to hijack user sessions even after the user logs out.                                            |
| Evidence        | Browser DevTools shows session cookie without Secure or HttpOnly attributes. Session remains valid after logout.                                                                                |
| Remediation     | Set Secure and HttpOnly flags on all session cookies. Implement server-side session invalidation on logout. Use SameSite=Strict to prevent CSRF. Regenerate session IDs on authentication.      |

**VUL-008    Sensitive Information Disclosure    MEDIUM**

|                 |                                                                                                                                                                                    |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Category        | Information Exposure                                                                                                                                                               |
| CVSS Score      | 5.3 / 10.0                                                                                                                                                                         |
| Description     | Error pages display full stack traces including PHP version, server path, database connection errors, and SQL query fragments. The robots.txt file discloses internal admin paths. |
| Business Impact | Disclosed technology versions allow attackers to target known CVEs. Exposed paths reduce reconnaissance effort significantly.                                                      |
| Evidence        | Triggering a 500 error reveals PHP 5.x version, MySQL version, and server file path in the browser.                                                                                |
| Remediation     | Configure custom generic error pages. Suppress PHP and server version headers (expose_php=Off in php.ini). Audit robots.txt — avoid listing sensitive paths.                       |

## 05 | Risk Classification Matrix

The following matrix defines the risk levels used throughout this report:

| Level    | CVSS Range | Business Meaning                                           | Action Required           |
|----------|------------|------------------------------------------------------------|---------------------------|
| CRITICAL | 9.0 – 10.0 | Immediate compromise likely; full system takeover possible | Fix within 24–48 hours    |
| HIGH     | 7.0 – 8.9  | Significant data exposure or service disruption possible   | Fix within 1 week         |
| MEDIUM   | 4.0 – 6.9  | Moderate risk; can aid or amplify other attacks            | Fix within 1 month        |
| LOW      | 0.1 – 3.9  | Minor risk; unlikely to be exploited alone                 | Fix in next release cycle |

## 06 | Top Remediation Priorities

|   |                                                                                                                                                     |
|---|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Fix SQL Injection immediately</b><br>Use prepared statements for ALL database queries. This single fix eliminates the most severe vulnerability. |
| 2 | <b>Enable HTTPS / TLS</b><br>Install a TLS certificate and redirect all traffic to HTTPS. Enable HSTS.                                              |
| 3 | <b>Add Security Headers</b><br>Configure CSP, X-Frame-Options, X-Content-Type-Options, and HSTS on the web server.                                  |
| 4 | <b>Fix XSS vulnerabilities</b><br>Sanitise all user input on server side; HTML-encode all output before rendering.                                  |
| 5 | <b>Secure Session Management</b><br>Set Secure + HttpOnly flags on cookies. Invalidate sessions on logout.                                          |
| 6 | <b>Disable Directory Listing</b><br>Set Options -Indexes in Apache / autoindex off in Nginx.                                                        |
| 7 | <b>Suppress Error Details</b><br>Configure custom error pages. Set expose_php=Off. Audit robots.txt.                                                |

## 07 | Conclusion

The vulnerability assessment of **testphp.vulnweb.com** revealed a high-risk security posture with one Critical and five High severity findings. The most dangerous issue — SQL Injection — could allow an attacker to gain complete access to all application data with minimal effort. Combined with the absence of HTTPS and security headers, this application represents significant business and reputational risk if deployed in a production environment.

The recommended remediation steps are practical, low-cost, and implementable by any development team. Addressing the Critical and High findings should be treated as urgent. A follow-up assessment is recommended within 30 days of implementing fixes to verify remediation effectiveness.

This report was prepared as part of the **Future Interns Cyber Security Internship Program – Task 1 (2026)**. Assessment was performed ethically using passive techniques only. No systems were harmed.